SECTION 5 | LECTURE 13

# 수업 13. 위험과 관련된 판단을 어떻게 통제할까?

심각도·정책·AI 추천의 구조화된 통제

# 리스크 통제의 시작: 품질을 통과한 결과도 그대로 사용할 수 있을까?

SOURCE 01. EXTERNAL INPUT

## 외부 요청 심각도

외부 클라이언트가 전달한 Incident severity를 검증 없이 실제 처리 기준으로 신뢰

```
// Client Request Payload
service: "payment-service"
severity: "MEDIUM"
```

외부 입력을 그대로 믿어도 될까?

SOURCE 02. POLICY DATA

## 텍스트 정책 데이터

서비스 최소 심각도와 에스컬레이션 정책을 문자열로만 전달하여 앱이 명시적으로 다루기 어려움

```
// Static YAML Policy
payment-service.min = HIGH
policyText = "Notify on-call DBA..."
```

정책을 단순 문자열로만 남겨도 될까?

SOURCE 03. AI OUTPUT

## 품질 통과 AI 추천

근거 일치성과 구체성은 높지만 시스템 상태를 변경하는 파괴적 액션이 무방비로 포함

```
// Quality Passed Action
actionTitle = "Restart the service"
target =  Production DB Cluster"
```

품질 통과 액션을 그대로 전달해도 될까?

> **핵심 구분:** 품질 통과(Quality PASS)는 신뢰성 있는 내용일 뿐, **안전성(Risk Controlled)까지 보장하지는 않습니다.**

Quality PASS ≠ Risk Controlled

# 위험 통제 아키텍처: 세 개의 명시적 통제 지점

CONTROL POINT 01 — INPUT GATE

## 외부 심각도 통제

클라이언트 요청값을 보존하면서 서비스별 최소 심각도 정책을 적용해 실제 처리값 산출

Requested Severity (요청값 보존)

↓ minimumSeverity 정책 적용

Effective Severity (실제 처리 기준)

외부 판단과 앱 최종 처리 판단 분리

CONTROL POINT 02 — POLICY GATE

## 에스컬레이션 정책 구조화

YAML 문자열을 그대로 평탄화하지 않고, 앱이 명시적으로 다룰 수 있는 결정 객체로 유지

YAML Policy Data (설정값 로딩)

↓ 도메인 객체 변환

Structured Decision (타입 결정)

Prompt 텍스트 변환 + 외부 응답 구조화 보장

CONTROL POINT 03 — OUTPUT GATE

## AI 추천 위험 통제

품질 통과 Action에 위험 분류, 의미 중복 제거, 정책 필터링을 거쳐 최종 노출 결정

AI Actions (품질 통과 추천 3개)

↓ 위험 분류 + 정책 적용

Final Actions (안전 추천 0~3개)

부족한 개수를 AI에게 재요청하지 않음

**공통 통제 원칙:** 입력이나 AI 출력을 그대로 최종 판단으로 사용하지 않고, **Java 애플리케이션의 명시적 통제 지점**을 둡니다.

* Deterministic Application Authority

# 실습 검증 시나리오: 오늘 실습에서 확인할 것

### STEP 01. IDE 통제 지점 코드 확인

Severity Policy, Structured Escalation, Recommendation Risk Control의 3대 통제 지점 구현 확인

### STEP 02. POSTMAN 심각도 및 정책 검증

requestedSeverity(MEDIUM)가 minimumSeverity 정책을 거쳐 effectiveSeverity(HIGH)로 승격되는 흐름 관찰

### STEP 03. UNIT TEST 추천 위험 분류 & 필터링

위험 키워드 분류, 의미 중복 제거, 정책상 제외 처리로 최종 액션이 0~3개로 제어되는 결정론적 테스트 검증

// RISK CONTROL VERIFICATION TARGETS — Deterministic Policy Enforcement

| | | |
|---|---|---|
| Requested vs Effective Severity | 외부 요청(MEDIUM) 보존 + 정책 적용(HIGH) 확정 | [ Postman 응답 필드 확인 ] |
| Escalation Decision Structure | 단순 텍스트 대신 targets, sla, requiresApproval 등 구조화 | [ DTO 결정 데이터 노출 ] |
| 0 to 3 Final Action Policy | 위험/중복 제외 시 빈 목록(0개) 허용, AI 재요청 배제 | [ 단위 테스트 정책 통과 ] |

⚠ **실습 핵심:** 위험한 결과를 잘 생성하는 AI를 만드는 것이 아니라, **어떤 입력과 출력이 와도 애플리케이션이 처리 방식을 결정하는 구조**를 확인합니다.

*IDE 실습 전환 준비

# 오늘 정리 & 다음 수업 안내

TODAY SUMMARY

Risk Control

## 위험 판단의 3대 통제 지점 완결

- **외부 심각도 통제:** requestedSeverity 보존 + minimumSeverity 정책으로 effectiveSeverity 결정
- **정책 데이터 구조화:** 문자열 평탄화 대신 EscalationDecision 객체로 다루고 필요 시 텍스트 변환
- **추천 위험 통제:** 품질 통과 Action에 위험 분류·중복 제거·정책 필터링을 거쳐 최종 0~3개 노출
- **통제 주체 확립:** 위험과 관련된 최종 판단은 AI가 아닌 Java 애플리케이션의 명시적 규칙이 결정

**통제 결론:** 외부 severity도 그대로 믿지 않고, 정책도 문자열로만 남기지 않으며, **AI 추천도 그대로 외부에 전달하지 않습니다.**

UP NEXT LECTURE

## 다음 세션 미리보기 (공통 정책 컨텍스트)

정책은 만들어졌지만, 분석·추천·평가·개선 등 여러 번의 LLM 호출마다 정책 컨텍스트를 일일이 따로 넣으면 누락되거나 어긋날 수 있습니다.

**Lesson 13 (위험 정책):** 어떤 조치를 허용하고 제외할 것인가? (Java 정책)

**Next Lesson (호출 컨텍스트):** 여러 LLM 호출 경계에 공통 정책을 어떻게 일관되게 주입할 것인가?

TRANSITION: RISK POLICY → ADVISOR CONTEXT 도메인 정책 판단을 AI에 넘기지 않고, 공통 정책 컨텍스트를 일관되게 적용하기

Move to Policy Context